Optimising Attention on a CPU

CS6886 Systems for Deep Learning — Assignment 5

AMD Ryzen 7 6800H (8C/16T Zen 3+), 16 GB DDR5-4800, Windows 11. Built with gcc 14.1.0 and `-O3 -march=native`.

I optimised the given attention kernel step by step on my laptop. At N=4096, d=64 the naive version takes 1375.9 ms and reaches 3.12 GFLOP/s. My final single-threaded version takes 47.8 ms (89.8 GFLOP/s), so it is 28.8× faster. With 16 threads it takes 10.6 ms (406 GFLOP/s), 130× faster than naive. All six variants pass `make check`.

1 My machine

The CPU is an AMD Ryzen 7 6800H. I read most of the specification from SMBIOS and CPUID, and I measured the rest with small probes in `tools/probe.c`. Table 1 has everything.

Table 1: My machine. "Measured" means I ran a microbenchmark, not that I copied a datasheet.

CPU	AMD Ryzen 7 6800H, Zen 3+ (family 0x19, model 0x44)
Cores / threads	8 physical, 16 logical, 2 CCX × 4 cores. No P/E split.
Clock	3.20 GHz base, 4.75 GHz rated boost. **I measured 4.29 GHz on one core, 3.64 GHz all-core.**
SIMD	AVX2 + FMA3, 256-bit, 2 FMA units per core, 16 YMM registers. No AVX-512.
Caches	L1D 32 KiB/core, L2 512 KiB/core, L3 16 MiB (8 MiB per CCX), 64 B lines.
Memory	16 GB DDR5-4800 SO-DIMM, 2 channels. DRAM latency 144 ns.

Zen 3+ does two 256-bit FMAs per cycle per core. One FMA covers 8 floats and counts as 2 FLOP, so a core does 32 FLOP per cycle. My FMA chain gives 4.29 GHz on one core:

$$P_1 = 32 \times 4.29 = 134.4 \text{ GFLOP/s}, \qquad P_8 = 32 \times 8 \times 3.64 = 932 \text{ GFLOP/s}$$

Over a 512 MB buffer I measured 25.8 GB/s on one thread and 36.0 GB/s on 16 threads, against 76.8 GB/s theoretical for DDR5-4800 dual channel. That gives

$$B_{\text{theor}} = 4800 \times 8 \times 2 = 76.8 \text{ GB/s}, \quad R_1 = 134.4/25.8 = 5.21 \text{ FLOP/B}, \quad R_8 = 932/36.0 = 25.87 \text{ FLOP/B}$$

One thing I did not expect: an earlier session on the same laptop gave 2.38 and 2.02 GHz instead of 4.29 and 3.64 GHz. The power plan and the supply were the same, so the platform seems to limit the clock by itself. I now re-measure the peak every session and use that value below.

2 The naive kernel

Table 2: Naive kernel. Median of 5 runs after one warm-up.

N/d	time (ms)	GFLOP/s	% measured peak	% rated peak
1024/64	79.02	3.40	2.5	2.2
2048/64	313.88	3.42	2.5	2.2
4096/64	1375.94	3.12	2.3	2.1
4096/128	3980.03	2.16	1.6	1.4

Effect of N. Going from 1024 to 2048 costs 4.0× and from 2048 to 4096 costs 4.4×, but the work only grows by 4×. The score matrix is the reason. At N=2048 it is 16 MB and still fits in the 16 MB L3. At N=4096 it is 64 MB, so every pass over it goes to DRAM.

Effect of d. Doubling d costs 2.9×, not 2×. In step 3 I walk j with stride d, so at d=64 a 64 B line gives me four floats, and at d=128 every load pulls a line and I use 4 bytes out of 64.

Where the time goes. I timed the three steps separately with `tools/naive_phases.c`. S V is 61 % of the time at 1024/64 and 76 % at 4096/128. I checked with `-fopt-info-vec` and gcc refuses to vectorise that loop, it says *"not suitable for strided load"*. So every element is a scalar load plus one step of a 4-cycle add chain. $QK^\top$ does get vectorised but only reaches 4.9 GFLOP/s, because one accumulator serialises the reduction. The softmax is only 1–4 % here, so I left it alone for now.

3 Optimisations

(a) Loop reordering. I transpose K once, then I change the loop order so the inner loop is an axpy instead of a dot product. An axpy has no loop-carried dependency, so gcc vectorises it without `-ffast-math`. For $QK^\top$ that means k outer and j inner, and for S V it means j outer and k inner. This gave 7.3×. What still hurts is that

Table 3: All my variants, one session. Each cell: time (ms) / GFLOP/s / % of measured peak / speed-up over naive. Every variant passes the check, error 1.1×10^{-6} to 5.0×10^{-6} against the 10^{-4} limit.

variant	1024/64	2048/64	4096/64	4096/128
`loops`	10.7 / 25.1 / 19 / 7.4	46.1 / 23.3 / 17 / 6.8	188.2 / 22.8 / 17 / 7.3	313.8 / 27.4 / 20 / 12.7
`tiled`	6.0 / 45.0 / 33 / 13.2	31.7 / 33.9 / 25 / 9.9	115.9 / 37.0 / 28 / 11.9	168.9 / 50.9 / 38 / 23.6
`fused`	8.2 / 32.8 / 24 / 9.6	34.0 / 31.6 / 24 / 9.2	131.4 / 32.7 / 24 / 10.5	295.4 / 29.1 / 22 / 13.5
`best`	3.0 / 88.7 / 66 / 26.1	11.5 / 93.7 / 70 / 27.4	47.8 / 89.8 / 67 / 28.8	92.3 / 93.1 / 69 / 43.1
`omp`	0.9 / 293.9 / 32 / 86.5	2.8 / 384.0 / 41 / 112.3	10.6 / 405.5 / 44 / 129.9	22.2 / 386.7 / 42 / 179.2
`naive`	79.0 / 3.4 / 3 / 1.0	313.9 / 3.4 / 3 / 1.0	1375.9 / 3.1 / 2 / 1.0	3980.0 / 2.2 / 2 / 1.0

nothing is blocked, so for every row I read all of $K^\top$ and all of V again out of L3, about 8 GB at N=4096. So this version is limited by L3 bandwidth, not by the FP units.

(b) Tiling. $QK^\top$ now uses a 4×16 micro-tile. That is 8 YMM accumulators with k innermost, so per k I issue 8 FMA (128 FLOP) against two 32 B loads and four broadcasts. 3 load cycles cover 4 FMA cycles, so the FMA units become the limit. For S V I block the reduction instead of the output: a 64-key slab of V is 16 KB and stays in L1 while every output row uses it, and that keeps the d accumulators in registers. Three things I had to learn by trying:

- gcc will not keep the tile in registers. In plain C it vectorises the wrong axis and spills. It only works when I write the tile with AVX2 intrinsics.
- A 16 KB row is a multiple of the L1 size, so a block of rows lands on one L1 set and thrashes. Padding each row by 64 B took this from 370.6 to 307.0 ms at 4096/64.
- One store per FMA caps a kernel at half of peak. My first S V kept the output tile in L1 and only reached 21 %. With the accumulators in registers it reaches 45 %.

Now I get 11.9×. The kernels split 22/44/34 % ($QK^\top$/softmax/S V) and $QK^\top$ runs at 71 % of peak. Two problems are left. The scalar `expf` is 44 % of the kernel time, and another 14 ms (12 % of the wall time) is not in any kernel at all, it is page faults from allocating the 67 MB score matrix every call.

(c) Fusion. Row i of O only needs row i of S, so I keep one score row in a 16 KB L1 buffer and run the three steps over it. The $N \times N$ matrix is never built. This gave 10.5×, which is slower than `attn_tiled` (131 against 116 ms) and much slower at d=128 (295 against 169 ms). I expected fusion to win, so I looked at the traffic. It saves about 200 MB of score matrix traffic but it makes every row read all of V, which is 4 GB of L3 traffic at d=64 and 8 GB at d=128. The traffic I add is bigger than the traffic I remove.

(d) Fastest single-threaded. I made two changes. First I block 64 queries, so the score tile stays in cache, there is no 67 MB allocation, and V is read once per block instead of once per row. Second I replaced `expf` with an AVX2 exponential: split $y = x \log_2 e$ into $n + f$ with $|f| \le 0.5$, evaluate 2^f with a degree-5 polynomial, and build 2^n by writing $n + 127$ into the exponent field. It is 8.5× faster than libm (16.8 M calls: 5.0 ms instead of 42.3 ms), and the whole kernel still stays within 5.0×10^{-6} of the naive result. Now I get 28.8× and 89.8 GFLOP/s, which is 67 % of the measured peak. That is 2.4× faster than `tiled` and 2.7× faster than `fused`.

4 OpenMP

I parallelised the query blocks. $K^\top$ is built once before the region starts. No barrier is needed: blocks write disjoint $64 \times d$ slices of O, everything they read is read-only, and the slices are 16 KB apart so there is no false sharing. One surprise: with `OMP_PROC_BIND=close` Windows puts both threads of a 2-thread run on the two SMT siblings of one core, and I got 53.9 ms instead of 45.3 ms. Table 4 is without binding.

Table 4: Strong scaling. Speed-up is against the same binary at one thread.

N/d	threads	time (ms)	speed-up	eff. (%)	GFLOP/s
	1	50.37	1.00	100	85.3
	2	25.29	1.99	100	169.8
4096/64	4	13.59	3.71	93	316.1
	8	10.83	4.65	58	396.5
	16	10.59	4.76	30	405.5
	1	91.82	1.00	100	93.6
	2	49.30	1.86	93	174.2
4096/128	4	27.13	3.38	85	316.6
	8	24.90	3.69	46	345.0
	16	22.21	4.13	26	386.7

Up to 4 threads it is close to linear. 8 threads give 4.65×, which is 58 % efficiency, and I think the clock explains the gap: all-core is 3.64 GHz against 4.29 GHz on one core, so the best I can get here is $8 \times (3.64/4.29) = 8 * 3.64/4.29 : .1f\times$. 16 threads give 4.76×. The kernel is FMA bound and the two SMT threads of a core share

the same two FMA pipes. The whole machine reaches 932 GFLOP/s at 16 threads against $8 \times 134.4 = 1075$ from the single core peak, so SMT adds about a quarter and not double. Efficiency is counted against thread count, so it drops fast even though the time still improves.

5 Roofline

I used the numbers I measured in section 1: 134.4 GFLOP/s at 25.8 GB/s on one core. Intensity is the useful FLOPs divided by the bytes a variant has to move. The numerator is always $4N^2d$; I did not count the softmax in the FLOPs. For the bytes I counted what each kernel actually needs:

$$(4Nd + 3N^2) \times 4 = 205.5 \text{ MB} \Rightarrow 20.9, \qquad 4Nd \times 4 = 4.19 \text{ MB} \Rightarrow 1024, \qquad (4Nd + Nd + \tfrac{N}{64}Nd) \times 4 \Rightarrow 59.4$$

which are (naive, loops, tiled), (fused) and (best, omp) in that order.

Table 5: Every variant at N=4096, d=64 against the roofline.

variant	AI (FLOP/B)	ms	GFLOP/s	% of relevant peak
`naive`	20.9	1375.94	3.1	2
`loops`	20.9	188.19	22.8	17
`tiled`	20.9	115.95	37.0	28
`fused`	1024.0	131.42	32.7	24
`best`	59.4	47.84	89.8	67
`omp`	59.4	10.59	405.5	44

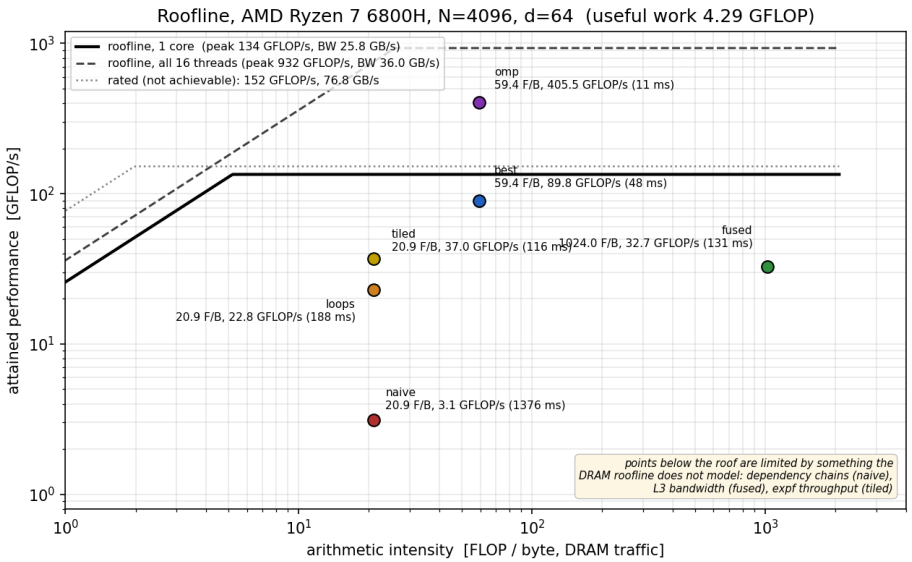


Figure 1: My roofline at N=4096, d=64.

What limits each one. `naive` is to the right of the ridge, so the roofline says it should reach 134.4 GFLOP/s, and I get 2.3 %. The roofline cannot explain that, because the strided loop costs one dependent add per element. `loops` is compute bound by the model and 5.9× below it, and its real limit is L3 bandwidth. `tiled` has $QK^\top$ at 71 % of peak, so what holds it back is `expf` and the page faults. `fused` is 197× past the ridge by the DRAM model and I only get 24 %, because its limit is L3 bandwidth, which is above DRAM and does not show on this plot. `best` reaches 67 % of peak and what is left is the softmax, S V at 45 % of peak, and reading V again. `omp` sits above the single-core roofline, which makes sense because it uses all the cores, and against the all-core roof it reaches 44 %.

How I did all this

The handout `Makefile` is unchanged. My laptop has no `make` (the MSYS2 one cannot start its signal pipe here) and no `clang`, so everything ran through `tools/bench.py`, which does the same steps with gcc 14.1.0 and runs the same check rule. `-march=native` picks `znver3`, the same ISA the Makefile would pick with clang. Every number is the median of 5 runs after one warm-up, and all tables come from one session because the clock moves. I only used `libm`, `immintrin.h` and `omp.h`. The driver in each file is untouched from `attn_naive.c`, and `results.csv` has 40 rows including every thread count.